

TU Dresden — Data Analytics & Distributed Computing

Distributed Computing I

Comprehensive Study Guide, Visual Notes & Exam-Style Q&A

Based on: *Digitization and Data Analytics — Lecture 10: Distributed Computing I* (TU Dresden / ZIH-CIDS). This guide expands the lecture slides into an in-depth revision resource: annotated diagrams, deeper conceptual explanations, comparative tables, and a research-oriented question bank in the style of MSc Computer Science examinations.

Contents

- 1. Why Distributed Computing? Motivation from the Data Pipeline
- 2. Core Definitions and System Architecture
- 3. Historical Evolution: Mainframe → DCC Systems
- 4. Big Data as the Driving Force (the V's, sources, motivation)
- 5. Scaling Strategies: Vertical vs. Horizontal
- 6. The MapReduce Paradigm and the Google File System
- 7. Hadoop, HDFS & the Hadoop 1.0 Ecosystem (incl. Hive)
- 8. Criticism, Limitations & the Big Data 2.0 Landscape
- 9. Data-Driven vs. Theory-Driven Analytics
- 10. Consolidated Summary Sheet
- 11. Exam-Style & Research-Oriented Q&A Bank (30 Questions)

1. Why Distributed Computing? Motivation from the Data Pipeline

The lecture frames distributed computing not as an isolated topic but as the **infrastructural answer** to a concrete problem: once data is digitized at scale, no single machine can collect, clean, integrate, analyze and interpret it fast enough. Distributed Computing (DC) = Data Analytics (algorithms/methods) + High Performance Computing (HPC, hardware/architecture) combined into a **Data Processing Pipeline**.

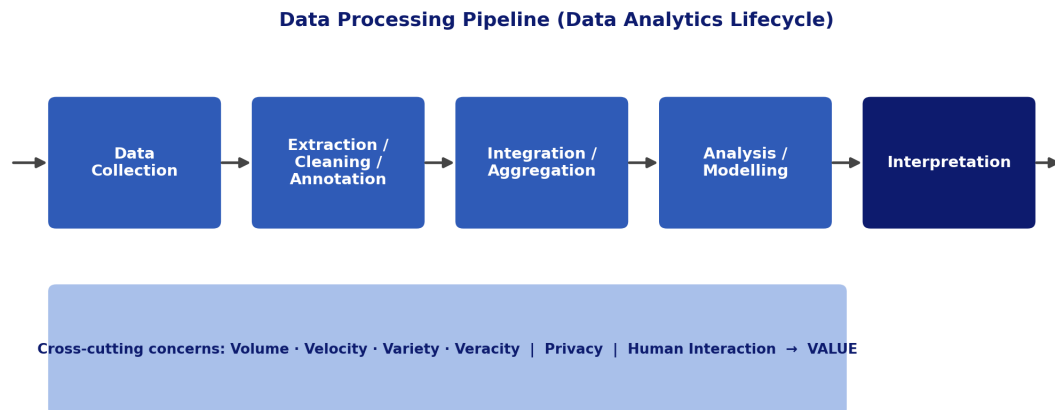


Figure 1 — The data processing pipeline, from raw collection to interpreted value.

Two complementary lenses are used throughout the lecture to describe the same underlying loop of data work:

(a) The data lifecycle (macro view): created/collected → processed → analyzed → published → archived → re-used → (feeds back into new data creation). The lecture highlights that *processed* → *analyzed* is the core computational bottleneck that distributed computing addresses.

(b) The five-stage pipeline (micro view): Data Collection → Extraction/Cleaning/Annotation → Integration/Aggregation → Analysis/Modelling → Interpretation, cross-cut by non-functional concerns (Volume, Velocity, Variety, Veracity, Privacy, Human Interaction) that all compete to be optimized simultaneously.

Key exam idea: distributed computing is motivated jointly by (i) hardware evolution (cheaper RAM/flash, faster networks, multicore) and (ii) new architectural/processing concepts — neither alone explains the shift; it is the co-evolution of both that enabled today's data-intensive pipelines.

1.1 Preprocessing Challenges (Garcia et al., 2015)

Before any distributed analysis can run, raw data typically fails five practical tests. Each challenge below maps to a concrete preprocessing technique:

Challenge	Why it arises	Technique
Need for meaningful features from raw data	Raw signals are rarely directly interpretable	Feature extraction
Analysis requires a specific data type/format	Algorithms expect fixed schemas or tensor shapes	Data portability
Erroneous, inconsistent or missing values	Sensors fail, forms are incomplete, sources disagree	Data cleaning
Data from multiple heterogeneous sources	Federated systems, multiple sensors/departments	Data integration
Analysis must remain computationally tractable	Curse of dimensionality, redundant records	Data reduction & transformation

2. Core Definitions and System Architecture

Distributed system (Tanenbaum & van Steen): a collection of autonomous computing elements that appears to its users as a single coherent system. **Distributed program**: a program that runs within such a system; writing it is *distributed programming*. The defining engineering trick is the **middleware layer**: a distributed-system software layer that hides heterogeneous local operating systems and hardware behind one uniform interface, so that applications see "one computer" instead of many.

Distributed System: Middleware Abstraction over Heterogeneous Machines

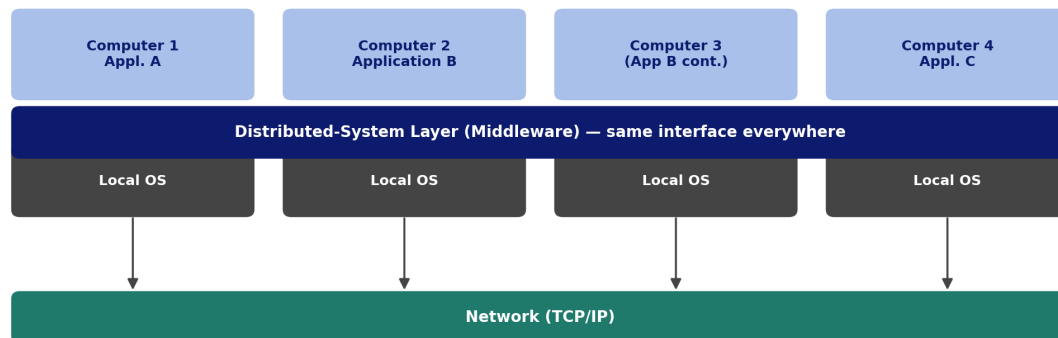


Figure 2 — Middleware provides the 'same interface everywhere' abstraction across nodes.

Distributing workload is a form of **parallel processing** — the lecture stresses that "concurrent computing", "parallel computing" and "distributed computing" are related but not identical: concurrency is about logically simultaneous execution, parallelism about physically simultaneous execution, and distribution additionally implies *no shared memory* and independent failure modes across nodes. Three canonical realizations were named:

- **Message passing** (e.g. MPI) — used in HPC for tightly-coupled, massively parallel simulations with explicit send/receive semantics and low-latency interconnects.
- **Grid computing** — loosely-coupled, often geographically distributed heterogeneous resources federated under shared middleware for large computational/data tasks.
- **Cloud computing** — virtualized, elastic, on-demand resource provisioning, abstracting hardware entirely behind a service interface.

Why this distinction matters for exams: MPI-style HPC optimizes for tightly synchronized, iterative numerical workloads (e.g. simulations) where communication latency is critical; Grid/Cloud optimize for loosely-coupled, embarrassingly parallel or service-oriented workloads where fault tolerance and elasticity dominate over raw communication speed.

3. Historical Evolution: Mainframe → Distributed Computing Continuum

Evolution of Distributed Computing Paradigms

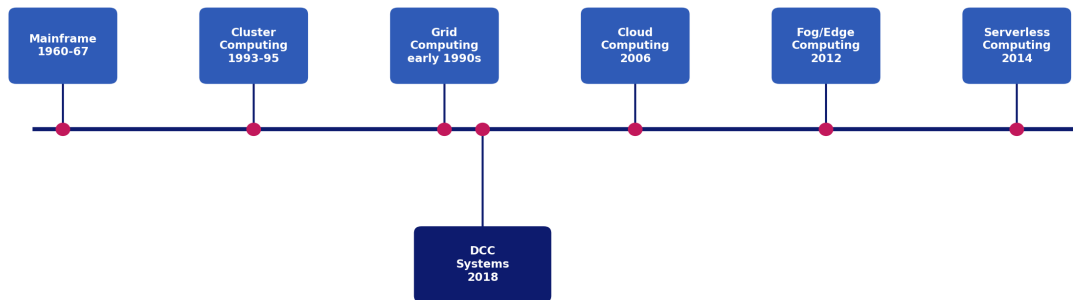


Figure 3 — Timeline of paradigm shifts (after Donta et al. 2023; Lindsay et al. 2021).

Paradigm	Strengths	Weaknesses / Trade-offs
Mainframe-based computing (1960-1967)	- Centralized, handles large workloads - Processes massive data reliably	- Expensive to build & maintain - Requires specialist skills
Cluster Computing (1993-1995)	- Cheaper & faster than mainframes - Fault tolerance & high availability	- Installation/maintenance complexity - Can fail to recover from data corruption
Grid Computing (early 1990s)	- Aggregates loosely-coupled heterogeneous resources - Designed for data-intensive apps	- Needs specialized middleware to manage - Involves many independently-owned resources
Cloud Computing (2006)	- No on-prem hardware needed - Elastic scaling, easy collaboration	- Privacy/security concerns - Higher latency, limited control over resources
Fog/Edge Computing (2009-2012)	- Processes locally → lower latency - Better privacy, less bandwidth use	- Constrained device resources - Energy constraints at the edge
Serverless Computing (2014)	- No infrastructure to maintain - Pay-per-use; scales automatically	- Cold-start & unstable performance - Limited control; vendor lock-in risk
Distributed Computing Continuum (DCC) Systems (2018)	- Seamless Cloud–Fog–Edge integration - Real-time processing + fault tolerance	- Interoperability across heterogeneous tiers - Security across the whole continuum

Alternative view (Lindsay, Gill & Smirnova, 2021) plots the same history on a *centralization* ↔ *decentralization* axis: Mainframe (1955, max. centralization) → Cluster (1962) → Network Computing (1967, packet switching/ARPANET) → Home Computer (1978) → WWW (1994) → Grid (1999) / P2P (1999, max.

decentralization) → Mobile Computing (2004) → Cloud (2006, virtualization/hypervisors) → IoT (2008) / SOA (2009) → Fog (2009) / Edge Computing (2009, infrastructure at the edge). The key challenges this history keeps re-solving are: **(1)** processing/analyzing massive data — scalability, data management, data security & quality; and **(2)** managing heterogeneous infrastructure — service orientation, fault tolerance, communication, security.

4. Big Data as the Driving Force

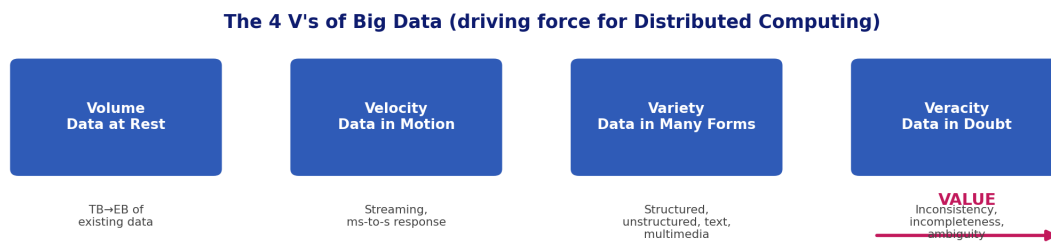


Figure 4 — The four defining V's of Big Data (a fifth, 'Value', is the outcome, not an input dimension).

Exam nuance: many textbooks list 5 V's by adding *Value* as a dimension; this lecture deliberately separates Value as the *goal* extracted from the other four — the slide states explicitly "more important: extract new content from database", i.e. Volume/Velocity/Variety/Veracity describe the *difficulty* of the data, while Value is what justifies the distributed-computing investment in the first place.

4.1 Where does Big Data come from?

- **Sensor data** (Brontobyte scale, 10^{27}) — IoT, scientific instruments (e.g. CERN ≈ 1 PB/second).
- **Mobile revolution** — smartphone shipments and mobile internet usage overtook desktop usage around 2013/14 (comScore data).
- **Event analysis** — logs, transactions, click-streams (Facebook ≈ 500 TB/day in the mid-2010s).
- **Consumer-generated content** — by 2013 $\sim 85\%$ of digital data was generated by consumers, but the majority of it (85%) was never 'touched' (processed) by enterprises — a persistent gap between data creation and data utilisation.

4.2 Business motivation to collect & analyze data

- Improving customer experience: service optimization, direct feedback, recommendation systems.
- Refining marketing strategy: quantifying response, 'understanding' the customer.
- Turning data into cash-flow: secondary data markets, sharing behavioral data between companies.
- Using data to secure data: e.g. voice/biometric signals in two-factor authentication.

Analytical goal: derive knowledge from data/information, then use 'analytics' (statistics, ML, or general AI methods) to uncover **correlations, causality, and systematic behavior** — note the epistemic distinction the lecture draws between correlation and causality, which becomes central again in Section 9.

5. Scaling Strategies: Vertical vs. Horizontal

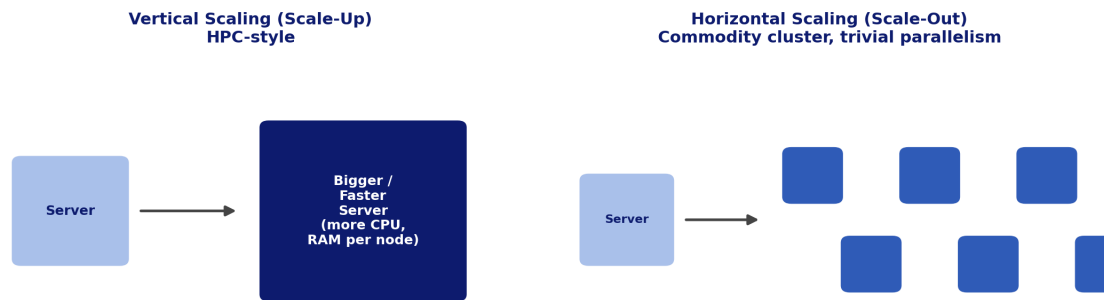


Figure 5 — Scale-up (fewer, more powerful nodes) vs. scale-out (many commodity nodes).

Aspect	Vertical Scaling (Scale-Up / HPC)	Horizontal Scaling (Scale-Out)
Typical use case	Massively parallel simulations, tightly-coupled iterative algorithms	Trivially parallel, partitionable, data-intensive workloads
Hardware assumption	One (or few) very powerful specialized machines	Hundreds–thousands of commodity servers
Communication pattern	Communication-intensive, low-latency interconnected nodes	More computation to data, minimal cross-node chatter
Failure handling	Assumes reliable hardware (single powerful node)	Assumes failure is the norm — must not rely on any single instance
Cost model	Expensive per unit of extra capacity (diminishing returns)	Roughly linear cost scaling by adding more cheap nodes

Ideal case for scale-out (verbatim principle from the lecture): "distribute computing to data, not vice versa" — i.e. ship the (small) program to where the (large) data already resides, rather than moving huge datasets across the network to a central compute node. This principle directly motivates HDFS's data-locality-aware scheduling in Section 7.

6. The MapReduce Paradigm and the Google File System

MapReduce emerged from two Google papers: **The Google File System** (Ghemawat, Gobioff, Leung — SOSP 2003) and **MapReduce: Simplified Data Processing on Large Clusters** (Dean & Ghemawat — OSDI 2004). Apache **Hadoop** is the open-source clone of this stack.

6.1 Programming Model

MapReduce decomposes a computation into two user-defined functions connected by a system-managed **shuffle** phase:

- **map(k1, v1) → list(k2, v2)**: applied independently and in parallel to each input split; performs filtering/transformation.
- **shuffle & sort**: the framework groups all values by key (k2) across the whole cluster — the only synchronization point in the model.
- **reduce(k2, list(v2)) → list(v3)**: aggregates all values for a given key, executed in parallel over distinct keys.

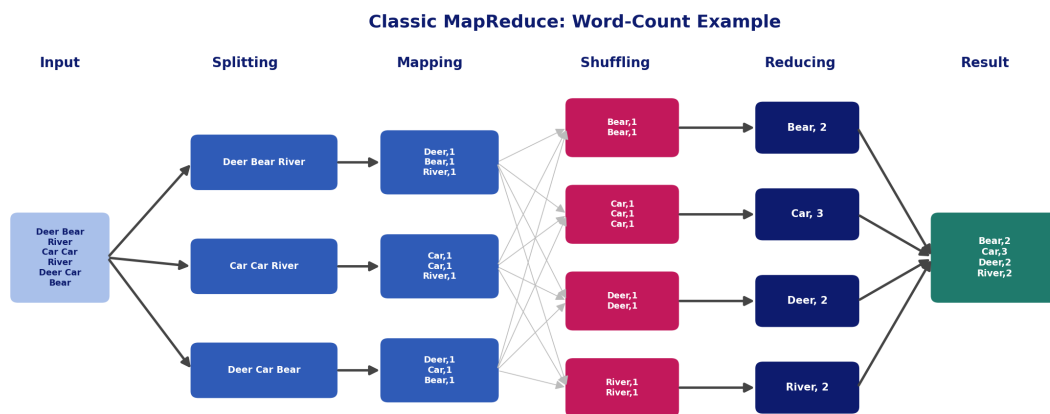


Figure 6 — Classic word-count MapReduce job: split → map → shuffle (sync point) → reduce → result.

Why word-count is the canonical teaching example: it cleanly separates the two 'pure' parallel stages (map, reduce) from the one unavoidable synchronization/communication stage (shuffle), which is exactly where distributed systems incur their main cost (network I/O, skew, stragglers).

6.2 HDFS: Hadoop Distributed File System

HDFS is Hadoop's storage layer (Apache-licensed v2.0, originally developed at Yahoo!, implemented in Java, typically on GNU/Linux). Large files (e.g. *MyBigFile*) are split into fixed-size **blocks** (B1, B2, B3, ...) which are the unit of both storage and replication.

HDFS: Block Replication Across a Shared-Nothing Cluster

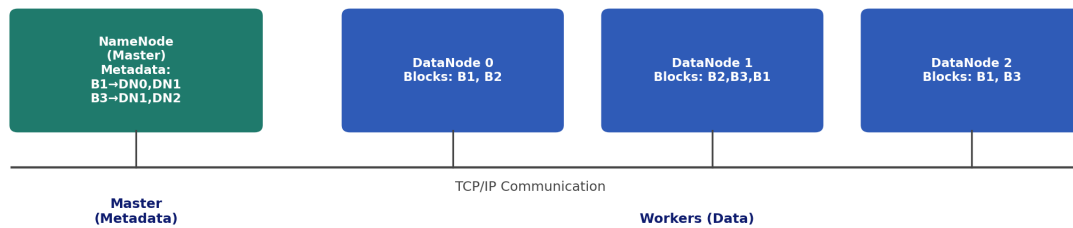


Figure 7 — NameNode (master, metadata only) coordinates DataNodes (workers, actual block replicas).

- **Blocks ≠ RAID:** HDFS keeps real, independent full-block *duplicates* across DataNodes rather than parity-based redundancy — simpler recovery, but higher storage overhead.
- **NameNode** holds only metadata (block→DataNode mapping); it is a classic single point of failure in Hadoop 1.0 (addressed later by HA NameNode / Hadoop 2.0).
- Blocks are the direct **input unit** for a MapReduce task — this is what lets Hadoop schedule computation next to data (data locality).

6.3 Design Assumptions and Goals (Hadoop/HDFS)

- **Hardware failure is the norm, not the exception** — hundreds to thousands of commodity servers per cluster.
- **Streaming data access**, not random low-latency access — optimized for batch processing over interactive queries.
- **Large data sets** — tens of millions of files, each gigabytes to terabytes.
- **Move computation to the data** — requires knowledge of data location (locality-aware scheduling).
- **Portability** across heterogeneous hardware and software platforms.

7. The Hadoop 1.0 Ecosystem & Apache Hive

Hadoop grew from 'just' MapReduce + HDFS into a layered, interoperable ecosystem:

Layer	Examples	Role
Coordination	ZooKeeper/Oozie	Workflow scheduling & cluster coordination
Workflow	Oozie	Chaining multi-step jobs into pipelines
Graph	Apache Giraph	Large-scale graph processing on Hadoop
SQL / Data Warehouse	Hive	SQL-like queries compiled to MapReduce jobs
Scripting	Pig	Dataflow scripting language over MapReduce
Distributed data processing model	Hadoop MapReduce	Core computation + job scheduling
Column store	HBase	Random-access NoSQL store on top of HDFS
Distributed storage	HDFS	Block-based replicated storage
Serialization	Avro	Cross-language data serialization format

7.1 Apache Hive — SQL on top of MapReduce

Hive = MapReduce + SQL. Motivation: SQL is simple and familiar, while still preserving MapReduce's horizontal scalability and fault tolerance. **HiveQL** is a SQL-like query language, so analysts do not need to hand-write low-level Java MapReduce jobs; HiveQL can still be extended with custom MapReduce scripts for non-relational logic.

Execution flow: (1) user issues a SQL query via CLI/JDBC/ODBC → (2) HiveServer2/Hive parses and plans the query (compiler, optimizer, executor) → (3) the query is translated into MapReduce job(s) → (4) Hadoop (NameNode, JobTracker, TaskTracker, DataNodes) executes the jobs and returns results.

Hive was originally developed at **Facebook** (first released 2010) and remains widely used (e.g. Netflix, FINRA). Amazon built its own Hive variant for Amazon Elastic MapReduce (EMR).

Example HiveQL word-count query (illustrating the SQL abstraction over MapReduce):

```
DROP TABLE IF EXISTS docs;
CREATE TABLE docs (line STRING);
LOAD DATA INPATH 'input_file' OVERWRITE INTO TABLE docs;
CREATE TABLE word_counts AS
SELECT word, count(1) AS count FROM
  (SELECT explode(split(line, '\s')) AS word FROM docs) temp
GROUP BY word
ORDER BY word;
```

8. Criticism, Limitations & the Big Data 2.0 Landscape

8.1 The 2008 DBMS-community critique of MapReduce

A well-known critique from the database research community (DeWitt & Stonebraker, 2008, referenced via 'MapReduce: A major step backwards') argued MapReduce was, from a DBMS perspective, a **step backwards** for large-scale data-intensive applications because it lacked core DBMS features and was incompatible with the existing DBMS tool ecosystem:

- **Indexing** — plain MapReduce has no index; it relies on 'brute force' full scans.
- **Updates** — no native mechanism to change data in place.
- **Transactions** — no parallel update semantics or recovery guarantees during updates (no ACID).
- **Integrity constraints & references** — no schema-level constraints as in relational databases.
- **Views** — no schema abstraction layer allowing the application to be shielded from schema changes.

Balanced exam answer: this critique is valid for MapReduce as a general-purpose data management system, but somewhat orthogonal to its actual design goal — massive, fault-tolerant batch processing over unstructured/semi-structured data at a scale where traditional RDBMS architectures did not scale horizontally at the time. Hive/HBase were later built precisely to reintroduce SQL-like semantics and random access on top of the Hadoop stack.

8.2 Further limitations of classic MapReduce

- **Non-interactive:** designed for batch jobs, not low-latency interactive queries.
- **Simple programming model, but complex to map real algorithms onto map/reduce** (e.g. iterative and graph algorithms are awkward to express).
- **No native iteration support:** each MapReduce round writes intermediate results to disk (HDFS) and re-reads them — very costly for iterative ML/graph algorithms.

Geoffrey Fox's classification (Indiana University) situates classic MapReduce as just one of six processing patterns: (1) Map-only, (2) Classic MapReduce, (3) Iterative MapReduce / Map-Collective, (4) Point-to-point / Map-Communication, (5) Map-Streaming, and (6) Shared-memory Map-Communicates — the last three progressively relax MapReduce's strict map→shuffle→reduce barrier to support iteration and low-latency communication, motivating **in-memory processing** frameworks like Spark.

8.3 From Hadoop (Big Data 1.0) to Big Data 2.0

Generation	Concern	Representative tech
200x — 1st generation	Volume dominant; batch reporting on logging data (few complex queries)	Hadoop, MapReduce, HDFS
201x — 2nd generation	Velocity & Value dominant; real-time processing, learning, analytics	Spark, Flink (in-memory); Storm, Samza (streaming)

The benchmark figure quoted in the lecture (Hadoop ≈110s vs. Spark ≈0.9s for an equivalent iterative in-memory workload) illustrates the huge performance gap for iterative algorithms once intermediate results stay in RAM instead of being re-written to HDFS every round.

Category	Frameworks
General purpose	Spark, Flink, Hadoop
Big SQL	Hive, Cloudera Impala, Presto, Spark SQL
Big Graph	Apache Giraph, GraphLab, GraphX
Big Stream	Storm, S4, Samza, Spark (Streaming), Flink

Three types of analytics scenario map onto these frameworks: **Batch processing** (ask a specific question of a bounded collection of data — classic MapReduce/Hive), **In-memory computing** (load all data into memory, run multiple operations, avoid staging data out — Spark/Flink), and **Stream processing** (continuously analyze changing data over time, where full history is often not needed — Storm/Samza/Flink).

9. Data-Driven vs. Theory-Driven Analytics (Why is this different from Scientific Computing?)

Classical scientific computing has a long history of **theory-based hypotheses**: a physical theory predicts a phenomenon, which is then confirmed by measurement/simulation — the textbook example given is General Relativity predicting gravitational waves, later confirmed by LIGO detecting merging neutron stars/black holes.

By contrast, many Big-Data domains (e.g. socio-economic systems) currently lack an equivalent predictive theory, forcing a **data-driven approach**:

- **Statistics**: derive global properties and probabilistic distributions directly from observed data.
- **General behavior of data distributions**: study empirical regularities without a mechanistic model.
- **Machine learning**: approximate functional dependencies — *supervised* (with an explicit hypothesis/label) or *unsupervised* (no hypothesis, purely data-driven pattern discovery).

New trend highlighted: fusing simulated/synthetic data with real measurement data:

- **Physics-informed neural networks (PINNs)**: ML models that incorporate real-world physical constraints directly into the model's state/loss function, rather than learning purely from data.
- **Digital Twins**: a digital copy of a real-world object/system that combines sensor data with a simulation/model, used for monitoring, prediction and what-if analysis.

Exam framing: distributed computing is the enabling infrastructure for both worlds — it lets theory-based simulations scale (HPC / vertical scaling / MPI) and it lets data-driven statistical/ML approaches scale over volume (Big Data stacks / horizontal scaling). The Digital Twin / physics-informed ML trend is where these two historically separate lineages are now converging.

10. Consolidated Summary Sheet

- Distributed Computing = connecting hardware & software capability to a new set of processing concepts, driven jointly by hardware evolution (cheap RAM/flash, faster networks, multicore) and new architectural ideas.
- The data-centric analytics need (Big Data's Volume/Velocity/Variety/Veracity) was the driving force behind Grid and Cloud Computing, and later Fog/Edge, Serverless, and DCC Systems.
- MapReduce's simple map → shuffle → reduce programming model gave ordinary developers access to large-scale distributed data sets without manual parallel-programming complexity.
- Adoption was accelerated by cheaper mass storage (disk, then RAM/flash) that made MapReduce's brute-force, replicate-everything approach economically viable.
- 'Big Data 2.0' platforms (Spark, Flink, Storm, ...) were developed specifically to overcome MapReduce's I/O bottleneck (materializing every intermediate result to disk) and to support genuine iteration, in-memory processing, streaming, and richer analytics chains beyond pure map/reduce.
- Two scaling philosophies persist side by side: vertical scaling (HPC, tightly-coupled, communication-intensive) vs. horizontal scaling (scale-out, commodity hardware, move-computation-to-data, assume failure is normal).
- The historical arc oscillates between centralization (mainframe, cloud) and decentralization (cluster, grid, P2P, edge) — modern DCC systems try to unify both ends into a single continuum.

11. Exam-Style & Research-Oriented Q&A Bank

The following 30 questions are written in the style of past TU Dresden MSc Computer Science exam/oral-exam questions for this lecture: definition questions, comparison questions, 'explain why' conceptual questions, and short research-oriented discussion questions. Model answers are concise but complete enough for full exam credit.

Q1. Define 'distributed system' and 'distributed program'. What role does middleware play?

Model answer: A distributed system is a collection of autonomous computing elements that appears to its users as a single coherent system (van Steen & Tanenbaum). A distributed program is any program that runs within such a system. Middleware is the software layer sitting between local operating systems and applications; it hides hardware/OS heterogeneity so every node exposes the 'same interface everywhere', which is what makes the collection of machines look like one system.

Q2. Distinguish parallel computing, concurrent computing and distributed computing.

Model answer: All three involve doing multiple things 'at once', but they differ in memory model and coupling: concurrent computing is about logically interleaved/simultaneous tasks (may run on one core via context switching); parallel computing physically executes tasks simultaneously, typically with shared memory (e.g. multicore, HPC/MPI); distributed computing explicitly assumes no shared memory, independent nodes, and independent partial-failure modes, coordinated via message passing over a network.

Q3. Name and briefly characterize three concrete realizations of distributed workload distribution mentioned in the lecture.

Model answer: (1) Message passing / MPI — used in HPC for massively parallel, tightly-coupled simulations requiring low-latency communication; (2) Grid computing — loosely-coupled heterogeneous resources federated by middleware for large data/compute tasks; (3) Cloud computing — virtualized, elastic, on-demand resources abstracted behind a service interface.

Q4. Order the following paradigms chronologically and give one defining property of each: Cloud Computing, Grid Computing, Cluster Computing, Serverless Computing, Fog/Edge Computing.

Model answer: Cluster Computing (1993-95, tightly-coupled homogeneous nodes acting as one system) → Grid Computing (early 1990s-1999, loosely-coupled heterogeneous resources run as a single system for data-intensive apps) → Cloud Computing (2006, elastic virtualized on-demand resources) → Fog/Edge Computing (2009-2012, processing pushed close to the data source to cut latency) → Serverless Computing (2014, no infrastructure management, automatic scaling of stateless functions).

Q5. What is a Distributed Computing Continuum (DCC) System and why was it introduced (Donta et al. 2023)?

Model answer: A DCC system integrates Cloud, Fog and Edge tiers into one seamless resource continuum so that workloads can be placed wherever is most efficient (latency, energy, bandwidth) rather than being locked into a single tier. It was introduced because pure Cloud (high latency, bandwidth cost) or pure Edge (constrained resources) each have complementary weaknesses; DCC systems aim for efficient resource utilization, seamless scalability, fault tolerance with real-time processing, and collaborative resource sharing — at the cost of harder interoperability and security-across-tiers challenges.

Q6. What are the four V's of Big Data described in the lecture, and how does 'Value' relate to them?

Model answer: Volume (data at rest, TB-EB scale), Velocity (data in motion, streaming, ms-to-s response), Variety (data in many forms: structured, unstructured, text, multimedia), and Veracity (data in doubt: inconsistency, incompleteness, ambiguity, latency, model approximation). Value is not a fifth raw-data property in this lecture's framing but the outcome: the whole point of managing Volume/Velocity/Variety/Veracity is to 'extract new content' — i.e., value — from the data.

Q7. Why is 'more important: extract Value from the data' a meaningful statement pedagogically?

Model answer: It reframes Big Data engineering away from purely technical scaling (store more, process faster) toward a business/science-driven question: scaling infrastructure is only justified if it produces usable knowledge or decisions. It reminds students that the 4 V's describe the difficulty of the problem, not the goal of the system.

Q8. List two business-driven and one security-driven motivation for collecting and analyzing large volumes of data.

Model answer: Business-driven: (1) improving customer experience through service optimization, feedback and recommendations; (2) refining marketing strategy by quantifying campaign response. Security-driven: using secondary behavioral/biometric data (e.g., voice) as an additional factor in two-factor authentication.

Q9. Explain the difference between vertical scaling and horizontal scaling, and state the 'ideal case' for each.

Model answer: Vertical scaling ('scale-up') increases the capability of a single system, typically by adding more/faster CPUs and RAM to fewer machines; its ideal case is highly parallel simulations with communication-intensive, iterative computation (HPC). Horizontal scaling ('scale-out') adds more instances of the same kind of (commodity) machine; its ideal case is distributing computation to where the data already resides, on separable/partitioned workloads with minimal need for cross-node communication.

Q10. Why does horizontal scaling explicitly assume 'failure is the norm rather than the exception'? What design consequence follows?

Model answer: With hundreds to thousands of commodity servers, the probability that at least one node fails at any given time is high, simply due to scale (statistically guaranteed hardware failures). The design consequence is that the system must never rely on any single instance: data must be replicated (HDFS block replication) and computations must be re-schedulable on failure (task re-execution), rather than relying on expensive fault-avoidance hardware as in vertical scaling/mainframes.

Q11. What two foundational Google papers underlie Hadoop, and what did each contribute?

Model answer: 'The Google File System' (Ghemawat, Gobioff, Leung, SOSP 2003) introduced a scalable distributed file system for large data-intensive applications running on commodity hardware — the design basis for HDFS. 'MapReduce: Simplified Data Processing on Large Clusters' (Dean & Ghemawat, OSDI 2004) introduced the map/reduce programming abstraction that let programmers express large-scale parallel computations without manually handling distribution, scheduling, and fault tolerance — the design basis for Hadoop MapReduce.

Q12. Describe the classic MapReduce programming model using the word-count example.

Model answer: Input text is split into partitions (Splitting). Each split is processed independently by a `map()` function that emits (word, 1) pairs (Mapping) — this stage is 'purely parallel', with no cross-node communication. The framework then groups all pairs by key across the whole cluster (Shuffling) — the only synchronization/communication-heavy stage. Finally, a `reduce()` function sums the counts per key in parallel (Reducing), producing the final word-count result.

Q13. Why is the shuffle phase described as the critical synchronization point in MapReduce?

Model answer: Map and reduce tasks are embarrassingly parallel and require no coordination between workers. Shuffle, however, must repartition and transfer intermediate key-value pairs across the network so that every value for a given key ends up at the same reducer — this all-to-all data movement is the main source of network I/O, skew and straggler effects in MapReduce jobs.

Q14. Explain HDFS block-based storage and why replication is done at the block level rather than at the file level.

Model answer: HDFS splits large files into fixed-size blocks (e.g. B1, B2, B3, ...), which are the unit of both storage distribution and replication; each block is duplicated across several DataNodes (real copies, not RAID-style parity). Block-level replication allows fine-grained load balancing and parallel access: different blocks of the same file can be processed concurrently by different map tasks, and a lost block only requires re-replicating that block, not the entire file.

Q15. What is the role of the NameNode vs. DataNodes in HDFS, and what classic weakness does this architecture have?

Model answer: The NameNode is the master: it stores only metadata (namespace, block-to-DataNode mapping) and coordinates access — it does not store actual data. DataNodes are workers that store and serve the actual block replicas and communicate with the NameNode over TCP/IP. The classic weakness of Hadoop 1.0 is that the NameNode is a single point of failure/bottleneck for metadata operations, later mitigated by High-Availability NameNode configurations in Hadoop 2.0+.

Q16. List the five HDFS design assumptions/goals from the lecture and explain the rationale behind 'move computation to data'.

Model answer: (1) Hardware failure is the norm; (2) applications need streaming (not random) data access, batch not interactive; (3) very large data sets (millions of files, GB-TB each); (4) move computation to the data; (5) portability across heterogeneous hardware/software. Moving computation to data avoids the network cost of transferring huge datasets to a central compute node — since data volumes vastly exceed program code size, it is far cheaper to ship the (small) map task to the node already holding the relevant data block.

Q17. Summarize the 2008 DBMS-community criticism of MapReduce and evaluate it critically.

Model answer: Critics (from the database systems community) argued MapReduce was a step backwards for large-scale data-intensive applications because it lacks indexing (relies on brute-force scans), native update/transaction support, integrity constraints, and view abstractions found in mature DBMSs. Critical evaluation: the critique is valid if MapReduce is judged as a general-purpose database, but MapReduce was designed for batch analytics over massive unstructured data at a scale relational systems of the time could not reach; subsequent systems (Hive, HBase) reintroduced SQL semantics and random access on top of the same scalable storage/compute substrate, addressing much of the original criticism without abandoning horizontal scalability.

Q18. What is Apache Hive, and how does it resolve the tension between SQL usability and MapReduce scalability?

Model answer: Hive is a data-warehouse-style layer on top of Hadoop offering HiveQL, a SQL-like query language. A user's SQL query is parsed/planned by Hive (compiler, optimizer, executor), translated into one or more MapReduce jobs, and executed by the underlying Hadoop cluster (NameNode, JobTracker/TaskTracker, DataNodes). This lets analysts use familiar SQL syntax while Hive automatically preserves MapReduce's inherent scalability and fault tolerance — no low-level Java MapReduce code is required for standard queries, though custom MapReduce scripts remain possible for non-relational logic.

Q19. Give the six MapReduce processing patterns identified by Geoffrey Fox and explain why iterative algorithms motivated new frameworks.

Model answer: (1) Map-only; (2) Classic MapReduce; (3) Iterative MapReduce / Map-Collective; (4) Point-to-point / Map-Communication; (5) Map-Streaming; (6) Shared-memory Map-Communicates. Classic MapReduce (pattern 2) materializes intermediate results to disk (HDFS) between rounds, which is very expensive for algorithms that iterate many times over the same data (e.g. iterative ML, graph algorithms, PageRank-style computations). Patterns 3-6 progressively relax the strict map→shuffle→reduce barrier to allow direct communication and in-memory state retention between iterations, motivating frameworks such as Spark and Flink.

Q20. Explain, with an approximate performance figure, why in-memory computing frameworks outperform classic Hadoop MapReduce on iterative workloads.

Model answer: The lecture cites a benchmark of roughly 110 seconds for Hadoop vs. ~0.9 seconds for Spark on an equivalent iterative task — a two-orders-of-magnitude speedup. The reason is architectural: Hadoop MapReduce writes intermediate results to HDFS (disk, replicated) after every map/reduce round and re-reads them for the next round, while Spark keeps data in distributed memory (RDDs) across iterations, avoiding repeated disk I/O and serialization overhead.

Q21. Categorize Spark, Flink, Hadoop, Hive, Presto, Apache Giraph, GraphX, Storm and Samza into the four 'Big Data 2.0' categories given in the lecture.

Model answer: General purpose: Spark, Flink, Hadoop. Big SQL: Hive, Cloudera Impala, Presto, Spark SQL. Big Graph: Apache Giraph, GraphLab, GraphX. Big Stream: Storm, S4, Samza, Spark (streaming), Flink (streaming).

Q22. Distinguish batch processing, in-memory computing, and stream processing as analytics scenarios.

Model answer: Batch processing asks a specific question of a bounded, already-collected bunch of data (classic MapReduce/Hive jobs). In-memory computing loads all relevant data into (distributed) memory and runs multiple operations on it, avoiding repeatedly staging data in and out of disk (Spark/Flink). Stream processing continuously analyzes data as it changes over time or arrives, where the full historical background is often not required for the answer (Storm/Samza/Flink streaming).

Q23. Contrast theory-based and data-driven approaches to scientific analytics, using the examples given in the lecture.

Model answer: Theory-based hypothesis testing starts from an established theory that predicts an outcome, which is then confirmed empirically — e.g. General Relativity predicting gravitational waves, later confirmed by LIGO observing merging neutron stars. Data-driven approaches are used where no comparable predictive theory exists (e.g. socio-economic systems): statistics derive global properties/probabilistic distributions directly from data, while machine learning approximates functional dependencies either with a hypothesis (supervised) or without one (unsupervised).

Q24. What is a 'physics-informed' neural network and a 'Digital Twin', and how do they relate to the theory-vs-data-driven distinction?

Model answer: A physics-informed neural network incorporates known real-world physical constraints directly into the model's state or loss function, rather than learning purely empirically from data — it hybridizes theory-based and data-driven approaches. A Digital Twin is a digital copy of a real-world object or system that fuses simulated/theoretical models with live measurement data. Both represent a convergence trend between the historically separate theory-driven (scientific computing) and data-driven (Big Data/ML) lineages of computational analysis.

Q25. Explain why 'hardware evolution' alone does not fully explain the rise of distributed computing, referencing the 2000-vs-now hardware/software comparison in the lecture.

Model answer: Around 2000, disk space was the cheap primary storage, network was costly, RAM was expensive, and single-core machines dominated — software was optimized for single-core object-oriented execution with SQL as the primary analysis language. Today, RAM/Flash has become cheap and fast enough to serve as primary storage, networks are much faster (aided by virtualization), and multicore/heterogeneous architectures dominate. But this hardware shift only became useful because of parallel software innovation — functional programming, multicore-programming and distribution frameworks, and NoSQL alternatives — which is why the lecture frames distributed computing as driven by hardware AND new processing concepts jointly, not hardware alone.

Q26. Why did 'Volume' dominate first-generation (200x) Big Data systems while 'Velocity' and 'Value' dominate the second generation (201x)?

Model answer: In the 200x era, only a few companies had genuine Big Data needs, batch processing/reporting was the dominant workload, and the biggest practical concern was simply storing and processing very large volumes (e.g., search indexes, basic logging/behavior analysis) — hence Hadoop's batch-oriented, HDFS-based design was sufficient. By the 201x era, most companies needed Big Data technology, use cases diversified into real-time processing and iterative learning workloads, so Velocity (fast/streaming response) and Value (usable outcomes) became the dominant concerns, which Hadoop's disk-bound batch model could not satisfy — motivating in-memory (Spark/Flink) and streaming (Storm/Samza) frameworks.

Q27. From a systems-design perspective, argue whether Grid Computing or Cloud Computing is better suited to a scientific consortium that needs to share heterogeneous, institution-owned compute resources for a shared large-scale experiment (e.g. particle physics data analysis).

Model answer: Grid Computing is generally the better fit: it was explicitly designed to federate large numbers of loosely-coupled, heterogeneous, independently-owned computing resources into a single logical system via specialized middleware, matching a multi-institution consortium model (e.g. the CERN Grid). Cloud Computing instead assumes a single provider offering elastic, virtualized, on-demand resources — well suited to commercial workloads with variable demand, but less naturally suited to a federation of independently governed institutional resources, and it introduces additional data-sovereignty and cost concerns for permanently large, continuously generated scientific datasets (e.g., CERN's ~1 PB/second sensor rate).

Q28. Discuss a key limitation of the DCC (Distributed Computing Continuum) model and propose one open research direction.

Model answer: A key limitation is ensuring seamless interoperability and compatibility across highly heterogeneous tiers (cloud servers, fog nodes, resource-constrained edge/IoT devices) while maintaining consistent security and privacy guarantees end-to-end. An open research direction is adaptive, context-aware workload placement/orchestration across the continuum (deciding at runtime whether a task should run at the edge, fog, or cloud based on latency, energy, and privacy constraints), combined with standardized security protocols that hold across trust boundaries between tiers.

Q29. Why does the lecture argue that 'Value', not Volume, is the ultimate justification for investing in distributed computing infrastructure? Give a counter-argument.

Model answer: The argument: collecting and storing ever-larger volumes of data is only worthwhile if it can eventually be turned into usable knowledge, decisions, or revenue (e.g. improved customer experience, new marketing insight, secondary data monetization) — otherwise organizations are simply paying storage/compute cost for data that is 'touched' by nobody (recall that ~85% of consumer-generated data was never processed by enterprises). Counter-argument: some Volume must be retained even without immediate identified value, e.g. for regulatory/compliance archiving, reproducibility of scientific results, or because future analytical techniques (not yet available) may later extract value from currently 'valueless' raw data — so investing in distributed storage purely for Volume can still be rational as an option on future Value.

Q30. Synthesize: explain how the data processing pipeline (Slide 4) and the historical evolution of distributed computing (Slide 8-11) are two views of the same underlying problem.

Model answer: The pipeline (Collection → Extraction/Cleaning → Integration → Analysis → Interpretation, cross-cut by Volume/Velocity/Variety/Veracity/Privacy/Human-Interaction) describes the recurring functional stages that any data-intensive system must perform. The historical evolution (Mainframe → Cluster → Grid → Cloud → Fog/Edge → Serverless → DCC) describes how the infrastructure available to execute that same pipeline has repeatedly been re-architected as data volume, velocity and variety grew beyond what the previous infrastructure paradigm could support economically or fast enough. In other words, the pipeline is the constant functional requirement, and the paradigm timeline is the changing technical answer to how to execute that pipeline at ever-increasing scale.

References (as cited in the source lecture)

- Wikipedia. *Distributed computing*. https://en.wikipedia.org/wiki/Distributed_computing
- van Steen, M., Tanenbaum, A.S. (2016). A brief introduction to distributed systems. *Computing*, 98, 967-1009. <https://doi.org/10.1007/s00607-016-0508-7>
- Donta, P.K., Murturi, I., Casamayor Pujol, V., Sedlak, B., Dustdar, S. (2023). Exploring the Potential of Distributed Computing Continuum Systems. *Computers*, 12, 198. <https://doi.org/10.3390/computers12100198>
- Lindsay, D., Gill, S.S., Smirnova, D. et al. (2021). The evolution of distributed computing systems: from fundamental to new frontiers. *Computing*, 103, 1859-1878. <https://doi.org/10.1007/s00607-020-00900-y>
- Garcia, S. et al. (2015). *Data Preprocessing in Data Mining*. Springer.
- Ghemawat, S., Gobioff, H., Leung, S. (2003). The Google File System. SOSP 2003.
- Dean, J., Ghemawat, S. (2004). MapReduce: Simplified Data Processing on Large Clusters. OSDI 2004.
- DeWitt, D., Stonebraker, M. MapReduce: A major step backwards. https://homes.cs.washington.edu/~billhowe/mapreduce_a_major_step_backwards.html
- Fox, G. Lecture: Building a Library at the Nexus of High Performance Computing and Big Data, Indiana University.
- Kleiner Perkins (2018). Internet Trends Report. <https://www.kleinerperkins.com/perspectives/internet-trends-report-2018>
- comScore (2014). The U.S. Mobile App Report.